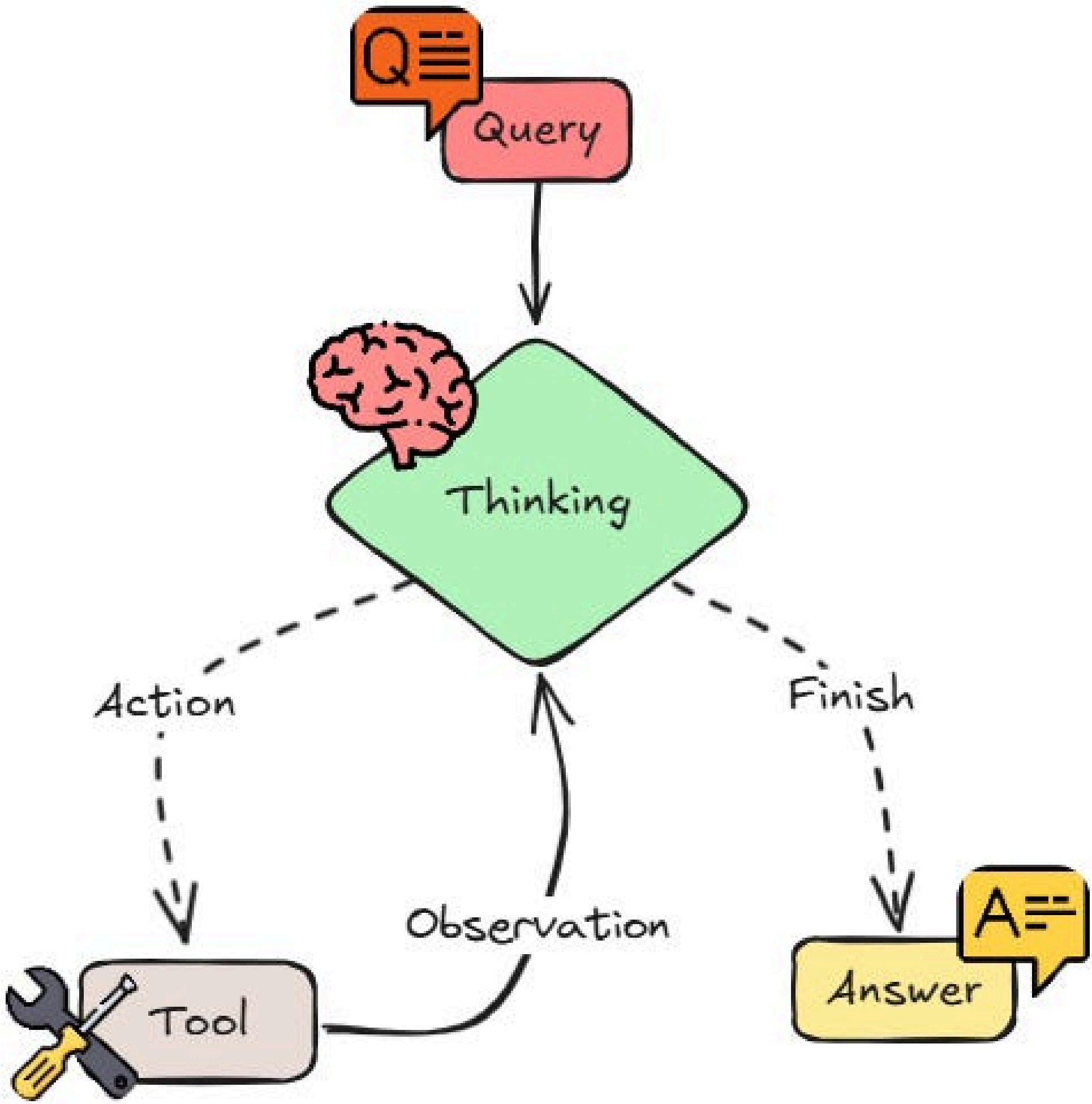
Analytics Vidhya

Complete Notes on ReAct Agents

Free Course



1. What is ReAct

ReAct stands for:

Reasoning + Acting

It is a prompting and agent framework where an LLM:

1. **Reasons about the problem**
2. **Chooses an action (tool use)**
3. **Observes the result**
4. **Repeats until the task is solved**

This loop allows the model to **think step-by-step while interacting with external tools**.

Core paper:

ReAct: Synergizing Reasoning and Acting in Language Models (2023)

---

2. Core Idea

Traditional LLM:

Input → Response

ReAct Agent:

```
Plain text

User Query
   ↓
Thought (reasoning)
   ↓
Action (tool call)
   ↓
Observation (tool output)
   ↓
Repeat until final answer
```

This enables:

- tool use
- real-time information retrieval
- multi-step reasoning
- dynamic decision making

3. ReAct Loop Structure

Standard loop:

```
Plain text

Thought → Action → Observation
```

Example:

```
Plain text

Question: Who is the CEO of Tesla?

Thought: I should search for Tesla CEO
Action: Search[Tesla CEO]

Observation: Elon Musk is the CEO

Thought: I now know the answer
Final Answer: Elon Musk
```

4. ReAct Prompt Format

Typical prompt structure:

```
Plain text

You are an intelligent agent that can reason and act.

Use the following format:

Question: input question
Thought: reasoning
Action: tool to use
Action Input: tool argument
Observation: tool result
... repeat ...
Thought: final reasoning
Final Answer: answer
```

5. ReAct Agent Architecture

```
Plain text

User
 ↓
LLM Reasoning
 ↓
Tool Selection
 ↓
Tool Execution
 ↓
Observation Returned
 ↓
LLM Reasoning
 ↓
Final Answer
```

Components:

Component	Purpose
LLM	reasoning engine
Tools	external capabilities
Memory	conversation history
Prompt	defines reasoning structure
Agent Executor	runs the loop

6. Core Components of ReAct Agents

1. LLM

Handles reasoning and decision making.

Examples:

- GPT-4
- Claude
- Mistral
- Llama models

---

2. Tools

Tools extend agent capabilities.

Examples:

Tool	Purpose
Search	retrieve web information
Calculator	math operations
Database query	retrieve structured data
Python execution	code execution
APIs	external services

---

3. Memory

Stores context and conversation history.

Types:

- short-term memory
- long-term memory
- vector memory

---

4. Agent Executor

Controls the ReAct loop.

Responsibilities:

- run reasoning steps
- execute tools
- track observations
- stop when task completed

7. ReAct Workflow

Step-by-step execution:

Plain text

```
1 User asks question
2 LLM reasons about next step
3 Agent selects tool
4 Tool executes
5 Observation returned
6 LLM updates reasoning
7 Loop continues
8 Final answer produced
```

8. Example ReAct Reasoning

Example question:

"What is the population of France divided by 2?"

Agent reasoning:

Plain text

```
Thought: I need France population
Action: Search[France population]

Observation: 67 million

Thought: Now divide by 2
Action: Calculator[67000000 / 2]

Observation: 33500000

Thought: I now know the answer
Final Answer: 33.5 million
```

9. ReAct Agent Implementation (LangChain)

Example implementation:

```python
from langchain.agents import initialize_agent
from langchain.agents import AgentType
from langchain.tools import Tool
from langchain_openai import ChatOpenAI

llm = ChatOpenAI()

tools = [
    Tool(
        name="Search",
        func=search_function,
        description="Useful for answering questions about current events"
    )
]

agent = initialize_agent(
    tools,
    llm,
    agent=AgentType.REACT_DESCRIPTION,
    verbose=True
)

agent.run("Who is the CEO of Tesla?")
```

---

10. ReAct with LangGraph (Modern Approach)

LangGraph provides **more reliable ReAct agents**.

Graph architecture:

```
User Input
   ↓
LLM Reasoning Node
   ↓
Tool Node
   ↓
Observation
   ↓
Loop until completion
```

Benefits:

- better control
- state management
- debugging
- production reliability

11. Tool Calling in ReAct

Tool format:

```python
def tool_function(input: str) -> str:
    ...
```

Tool metadata:

Field	Purpose
name	tool identifier
description	helps LLM choose tool
parameters	tool inputs

12. Stopping Conditions

ReAct loop stops when:

- LLM outputs **Final Answer**
- max iterations reached
- tool execution fails

Typical limit:

```
max_iterations = 10
```

---

13. Advantages of ReAct Agents

Improved reasoning

Step-by-step thinking improves accuracy.

Tool integration

Agents can interact with real systems.

Dynamic decision making

Agent adapts to observations.

Reduced hallucination

External tools verify information.

---

14. Limitations of ReAct

Slow execution

Multiple reasoning steps increase latency.

Token cost

Reasoning steps consume tokens.

Tool misuse

LLM may call incorrect tools.

Error propagation

Bad observation leads to wrong reasoning.

15. Optimization Techniques

Improve ReAct performance using:

Tool descriptions

Clear descriptions improve tool selection.

Few-shot examples

Demonstrate reasoning format.

Iteration limits

Prevent infinite loops.

Tool routing

Use routers to select relevant tools.

16. ReAct vs Other Agent Frameworks

Framework	Description
ReAct	reasoning + acting loop
Plan-and-Execute	plan first, then execute
Reflexion	agents self-evaluate
Tree-of-Thought	multiple reasoning paths
Graph Agents	structured workflows

17. Modern ReAct Agent Stack (2025–2026)

Typical production stack:

```
Plain text

User
 ↓
ReAct Agent
 ↓
LLM (GPT / Claude / Llama)
 ↓
Tools
 ↓
Vector DB
 ↓
External APIs
```

Common technologies:

- LangChain
- LangGraph
- OpenAI tools
- CrewAI
- AutoGen

18. Applications of ReAct Agents

AI copilots

coding assistants

research assistants

web search + summarization

data analysis agents

SQL + Python execution

customer support bots

knowledge base retrieval

automation agents

API orchestration

19. Best Practices

Use ReAct effectively:

1. design clear tools
2. keep tool descriptions precise
3. limit reasoning steps
4. add guardrails
5. log reasoning traces
6. monitor agent performance

20. ReAct + RAG Integration

Agents combine retrieval with reasoning.

Pipeline:

```
Plain text

User Query
   ↓
Retriever
   ↓
Context
   ↓
ReAct Agent
   ↓
Tools
   ↓
Answer
```

Benefits:

- grounded answers
- less hallucination
- dynamic retrieval

21. Evaluation Metrics

Measure agent performance using:

Metric	Description
task success rate	tasks completed
reasoning steps	efficiency
tool accuracy	correct tool usage
latency	response time
hallucination rate	incorrect facts

22. Security Considerations

Potential risks:

- prompt injection
- malicious tool inputs
- API abuse
- data leakage

Mitigation:

- input validation
- tool restrictions
- sandboxed execution
- output filters

---

23. ReAct vs Function Calling

Feature	ReAct	Function Calling
reasoning steps	explicit	hidden
tool usage	iterative	direct
transparency	high	medium
complexity	higher	lower

---

24. Future of ReAct Agents

Current research trends:

- multi-agent ReAct systems
- autonomous task decomposition
- long-term memory agents
- self-improving agents
- agent societies

Quick Mental Model

```
Plain text

Think → Act → Observe → Repeat
```

or

```
Plain text

Reasoning
   ↓
Tool Use
   ↓
Observation
   ↓
Improved Reasoning
```